

Streamable HTTP

Stateless

Deterministic

No external API

```

graph TD
    A[MCP Host 키] --> B[HTTP 요청 수신]
    B --> C[Cookie 처리 설정]
    C --> D{Cookie 정보와 키가 일치하는지}
    D --> E[OPTIONS]
    D --> F["GET /health/ <br/> GET / <br/> GET /api/ JSON 응답"]
    D --> G["POST /map <br/> JSON body 요구"]
    D --> H["GET /api/delete /map <br/> 405 Method not allowed"]
    D --> I[오류]
    E --> J[204 응답]
    F --> K["200 OK <br/> JSON 응답"]
    G --> L["200 OK <br/> JSON body"]
    H --> M[405 Method not allowed]
    I --> N[404 Not found]
    D --> O{본문 처리 성공?}
    O --> P[성공]
    O --> Q[실패]
    Q --> R["400 JSON RPC param error"]
    Q --> S["400 Map Server 생성"]
    R --> T[analyzePongBack 블록]
    S --> U[getReportChannels 블록]
    T --> V[StreamableHTTP transport 생성]
    U --> V
    V --> W[secondGenerator undefined]
    W --> X[status는 항상 처리]
    X --> Y[이동 블록 또는 400 응답 생성]
    X --> Z["close <br/> transportServer"]
  
```

[illegible]

```

graph TD
    A[current IQR] --> B{personality < 0.5?}
    B -- Yes --> C[situation <= 1232, 1204]
    B -- No --> D{personality < 0.25 or personality > 0.75?}
    D -- Yes --> C
    D -- No --> E{personality < 0.25?}
    E -- Yes --> C
    E -- No --> C
  
```

Figure 1: A flowchart illustrating the decision logic for the 'personality' variable. The process starts with 'current IQR'. A decision diamond asks 'personality < 0.5?'. If 'Yes', it leads to 'situation <= 1232, 1204'. If 'No', it leads to another decision diamond 'personality < 0.25 or personality > 0.75?'. From this second diamond, a 'Yes' path leads to 'situation <= 1232, 1204' and a 'No' path leads to a third decision diamond 'personality < 0.25?'. The 'personality < 0.25?' diamond has a 'Yes' path leading to 'situation <= 1232, 1204' and a 'No' path leading to 'situation <= 1232, 1204'.

[illegible]

```

graph TD
    Start([test code: getReportContent()]) --> Init[situation: 0과 1의 범위]
    Init --> Decision{situation}
    Decision -- suspectOnly --> SuspectOnly[이전 음향만 사용]
    Decision -- alreadyUsed --> AlreadyUsed[이전 음향 사용됨]
    Decision -- personnelExposed --> PersonnelExposed[개인정보 노출됨]
    Decision -- malwareInstalled --> MalwareInstalled[악성코드 설치됨]
    
    SuspectOnly --> SuspectOnlyData[1394, 118]
    SuspectOnly --> SuspectOnlyNote[최근만 사용 가능]
    
    AlreadyUsed --> AlreadyUsedData[112, 114, 115, 116, 117, 118, 119, 120, 121, 122, 1332, 1384]
    AlreadyUsed --> AlreadyUsedNote[최근 음향도 사용 가능]
    
    PersonnelExposed --> PersonnelExposedData[0과 1의 범위]
    PersonnelExposed --> PersonnelExposedNote[개인정보 노출을 노골]
    
    MalwareInstalled --> MalwareInstalledData[0과 1의 범위]
    MalwareInstalled --> MalwareInstalledNote[악성코드 설치됨]
    
    SuspectOnlyData --> Merge(( ))
    AlreadyUsedData --> Merge
    PersonnelExposedData --> Merge
    MalwareInstalledData --> Merge
    
    Merge --> Malware[무엇인지 & Malware]
    Malware --> Extract[이름을 추출하기]
    Extract --> Output[24bit byte 출력하기]
  
```

- 서버는 요청마다 새 MCP 서버와 transport를 만들고 닫는 **stateless 구조**다.
- 외부 API 호출 없이 정적 데이터와 규칙 엔진으로 **결정적 결과**를 만든다.
- 분석은 마스킹 → detectSignals(탐지용 정규화 → 신호별 정상 문맥 억제 → 패턴 매칭) → context 보정 → 점수화 → Markdown 포맷 순서다.
- 낮은 결과는 과한 경고를 생략하고, 주의 이상 결과는 **30초 안전 브레이크**로 시작한다.
- 이미 한 행동은 위험도 점수보다 **situation 선택**, 행동 가이드, 신고 루트 분기에 반영된다.
- getReportChannels는 분석 없이 상황별 공식 신고 채널만 반환하는 보조 도구다.